

AI Internship Case Study : Vaijayanti Deshmukh

Since due diligence happens before a deal memo, I focused on a 'Deal Memo Auto-Generator' because it leverages RAG to solve the high-friction transition between raw data and internal reporting, while maintaining the strict audit trail required in M&A. I've designed it to be implementable as a v1 using Python and LLM-agentic logic. I have made sure that we use Ethics of AI principles like Fairness, Accountability and Transparency to make our agent more responsible and ethical.

1) Workflow Choice

- Workflow Name: **Deal Memo Auto Generator (DMAG)**
- Why high-leverage:
 - We can help analysts to reduce the memo drafting time from hours to minutes.
 - It shall help analysts to focus on analysis rather than formatting.
 - It shall eliminate human data entry errors as we will be pulling out our data directly from verified sources(due diligence documents)
 - It shall give out a citation to make sure that all numbers are coming from due diligence.
- Primary User: Associate / Analyst

2) Goal

- What it automates: It automates the due diligence into a structured investment memo.
- What it **does not automate**: Does not give final judgement. Does not give any risk assessment.
- Human Process: Fits after the bulk of due diligence documents are collected and before the final internal investment committee review.

3) Inputs

- Provided Folder Docs: These can be meeting transcripts, financial reports in PDF/Word, CSV files
- Structured Data: We can use docx. We call it memo_template.docx
- Optional Web Enrichment: Cross-referencing LinkedIn/G2 data to verify market positioning claims found in the CIM.

4) Outputs

- Format: Editable docx, appendix of citations and a JSON metadata file for CRM integration.
- Schema Snippet: JSON

```
{
  "output_type": "memo",
  "sections": [{
    "title": "Executive Summary",
    "content": "...",
    "citations": [{"doc": "Pitch_Deck.pdf", "page": 3}],
    "confidence_score": 0.85
  }],
  "flags": ["Revenue discrepancy: CIM vs. Tax Filing"]
}
```

5) Step-by-Step Workflow

1. Ingest & Parse: Load all DD materials; utilize pandas for excel and csv extraction. We can use Tabula, Camelot to pull out tables from pdf. They turn messy tables into JSON. We can use PyMuPDF for pdf to text conversion as well.
2. Semantic Chunking: Index documents into a database. We can use any database or use vector embeddings and tag by logical sections like financials, market, team, etc. We can utilize hierarchical chunking to preserve the relationship between financial headers and their corresponding data.
3. Template Mapping: Parse the memo_template.docx to identify mandatory headers and content blocks.
4. Agentic Synthesis: For each section, trigger a RAG query to pull the top 5 relevant chunks and generate 2-3 paragraphs.
5. Financial Auto-Fill: Map specific metrics from DD into the memo's summary tables using Pydantic for validation.
6. Fact-Check & Reconcile: Run comparisons to flag any contradictory figures between different source documents.
7. Evidence Appendix: Compile an appendix that maps every inline citation to the specific quote in the source folder.
8. Export: Use python-docx to generate the final editable Word document for the analyst.

6) Quality & Safety

- Evidence Gating: The system is closed-book. If the AI cannot find a direct quote or specific page reference in the folder, it is blocked from answering to prevent hallucinations.
- Conflict Detection: Automatically flags discrepancies between documents (e.g., if the Pitch Deck and the Tax Return show different Revenue figures).
- Human-in-the-Loop: Any section with a low search match score is highlighted in red, forcing a mandatory manual review by the Analyst.

7) Evaluation

1. Human Acceptance Rate: % of the AI-generated text that the Analyst keeps without making changes (Goal: >70%).
2. Citation Accuracy: % of auto-generated citations that correctly point to the right page and document (Goal: 100%).
3. Speed-to-Market: Total reduction in hours from "Documents Received" to "First Draft Completed" (Goal: 75% time savings).

Tools Used: I drafted the initial architecture diagram (attached below) independently. I then used **Google Gemini** to assist with detailed logic flows and to select the optimal libraries for implementation

